

Project: Mini-Project

Milan Lagae

Institution/University Name:

Faculty:

Course:

Vrije Universiteit Brussel

Sciences and Bioengineering Sciences

GPU Computing

Contents

1	Intro	1
2	Structure	1
3	Methodology	1
4	Part 1: Addition vs Multiplication	2
4.1	Setup	2
4.2	GPU Analysis	2
4.3	CPU vs GPU	3
4.4	Conclusion	3
5	Part 2: Elements Per Thread	4
5.1	Setup	4
5.2	Continuous vs Strided	4
5.3	Conclusion	5
6	Part 3: Roofline Model	6
6.1	Setup	6
6.2	Model	6
7	Part 4: Local vs Global	7
8	Extra: Desktop vs Macbook	8
9	Appendix	9
9.1	Part 1	9
9.2	Part 2	11
9.3	Part 3	11
9.4	Specifications	11
9.4.1	Macbook	11
9.4.2	Desktop	12
	Bibliography	13

1 Intro

Throughout all parts of the mini-project, it was chosen to compare two **operations** namely: addition & multiplication. To start, a comparison between the operations will be done, as requested in part 1. Following that, the kernel will be updated to execute on multiple elements simultaneous (one-to-many) mapping.

For the third part, the compute intensity will be increased by applying a loop count factor over the computations.

The final part will compare the local vs global memory location, as indicated in the scheme¹ in the assignment.

2 Structure

TODO

3 Methodology

For each iteration of the combination of values to be measures, **20** runs where measured. The first 5 runs where discarded to allow the system to stabilize. This is in accordance to the recommendations in [1]. For each measured value, if applicable a Cov range chart is produced and will be reference, these can be found in §9. The acceptable range of the Cov is taken from [2].

¹http://parallel.vub.ac.be/education/gpu/labs/miniproject_step4_with_local%20memory.jpg

4 Part 1: Addition vs Multiplication

In this part, a comparison between the addition & multiplication operation on gpu's is made. While also comparing the general gpu performance to cpu performance.

4.1 Setup

The code for this part can be found in the file: part-1.cpp and the kernel file in partOne.cl.

4.2 GPU Analysis

Let's start of with analyzing the gpu performance. The easiest chart to get started, is time chart. Plotting the time vs the array size can be seen in Figure 1. With the color, differentiating between the 2 operations.

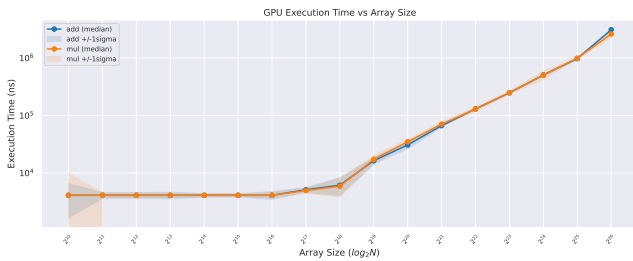


Figure 1:

The only notable, to remark on the plot, is that only starting from an array size of 2^{16} , does the computational intensity increase enough for the work to be reflected in the time taking. Starting from this point one, the time logarithmically with the size of the array.

Continuing from the time variable, to the memory bandwidth vs array size in Figure 2.

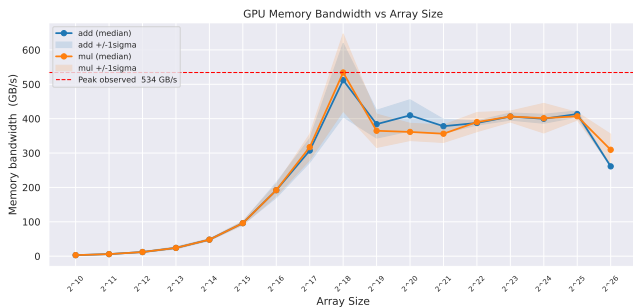


Figure 2:

The memory bandwidth increases with the increase in array size, with a peak at the array size of 2^{18} .

Increasing the size of the array beyond this 'ridge point', we are limited by the kernel (construction inefficiency) and available memory bandwidth. Increasing the array size to 2^{26} and beyond, showcase that the gpu **may** potentially be memory limited.

Before making strong conclusion's, let's take a look at the computationally intensity of the benchmark in Figure 3.

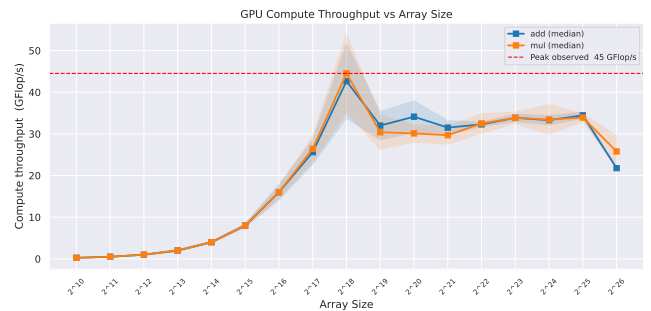


Figure 3:

The same curve as in Figure 2 is visible. The initial conclusion that can be drawn from this figure and the previous one, is that at the moment, this kernel is memory bound.

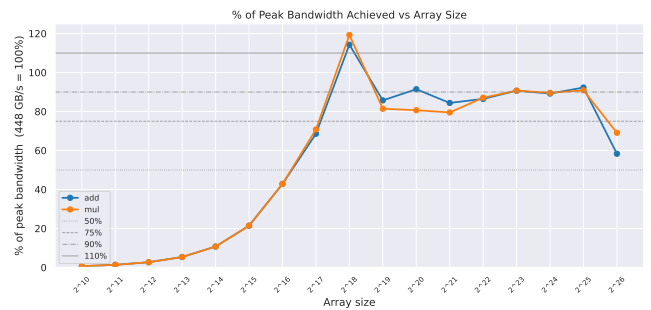


Figure 4:

When plotting the memory bandwidth in percentage to the max theoretical performance, it is clearly visible that the GPU is memory bound. The peak at array size of 2^{18} , could be explained by the fact the two source arrays and target arrays can be stored inside of the gpu's L2 cache, which

for an RTX 3070 is 4MB^2 .

4.3 CPU vs GPU

Let's start by comparing the GPU performance to CPU performance, this graph is depicted in Figure 5.

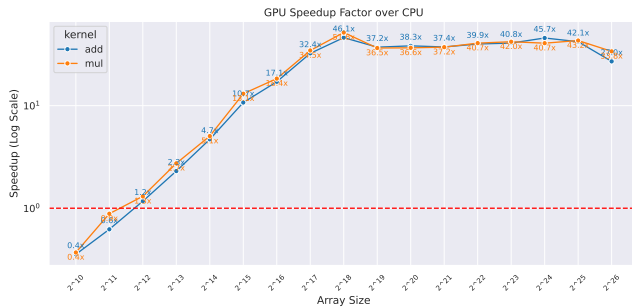


Figure 5:

Comparison to CPU performance is done using, a very naive CPU algorithm which does a loop over the loop. A better comparison, would be the usage of parallelism & potential even SIMD to further increase CPU performance. For this part, the CPU will keep using a naive algorithm.

As is visible on figure Figure 5, the speed gained by using the parallelism of the GPU has a massive impact on the performance between both.

4.4 Conclusion

From the conducted benchmarks and resulting graphs, it can be concluded that there is not considerably different in speed between the additional or multiplication operation on a RTX 3070 gpu. The comparison of the GPU to a naive CPU algorithm, showcase the massive parallelism capabilities of GPU's.

²<https://www.techpowerup.com/gpu-specs/geforce-rtx-3070.c3674>

5 Part 2: Elements Per Thread

This section will analyze the result of increasing the number of elements a single thread (kernel item) is responsible for on the performance of the execution. These different access patterns are described as continuous & strided. They are based on the pdf³ mentioned in the assignment description.

5.1 Setup

The array size is fixed for all benchmark variations performed in this part to 2^{22} , based on the data gathered in part 1 and the workgroup size is set to 64.

5.2 Continuous vs Strided

Start of with looking at the time it takes between both operations and different access patterns in figure Figure 6.

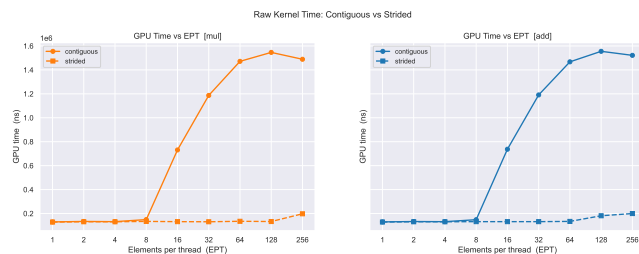


Figure 6:

This chart gives the first indication that the strided access pattern has better performance compared to the continuous pattern.

When putting the continuous pattern vs the strided pattern, and plotting the speedup in figure Figure 11, the picture becomes clearer.

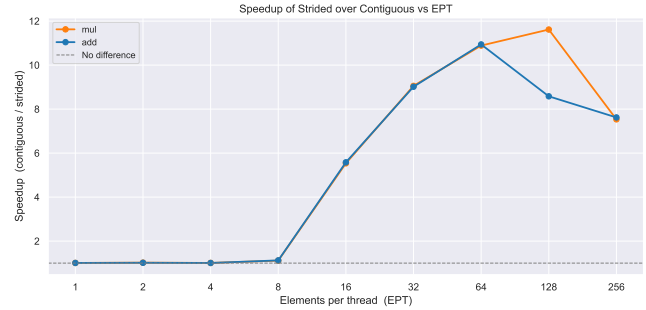


Figure 7:

The graphs show a very clear speedup of the strided access pattern starting from 8 ETP value. The reason for this speedup of this pattern becomes clear when looking at the memory bandwidth for the patterns in Figure 8.

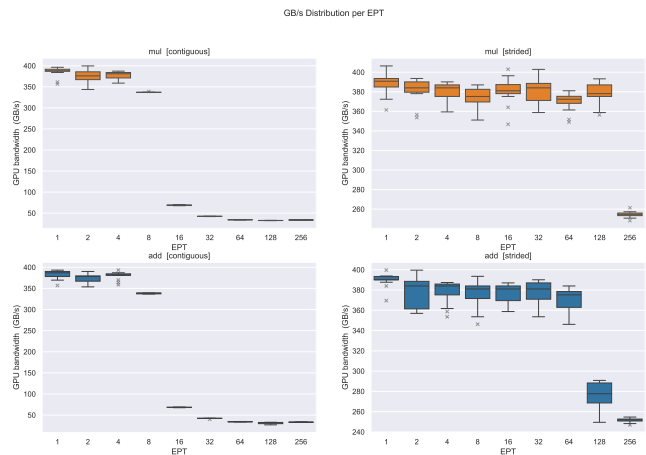


Figure 8:

The continuous access pattern is shown on the left and the strided on the right. The large drop in memory bandwidth for the continuous pattern when the EPT value is increased from 8 to 16 is noticeable. A similar drop in bandwidth is remarked for the continuous pattern when going from 64 to 128 and to 256.

Let's continue to take a look at the compute intensity of both strategies, as shown in Figure 9.

³<http://parallel.vub.ac.be/education/gpu/theory/GPU%20Computing%20-%20Lesson%202%20doc%20-%20Programming%20GPUs%20-%20levels%200%201%20and%202.pdf>

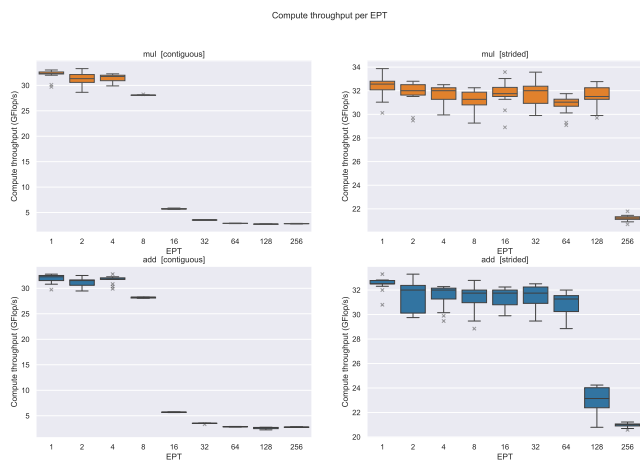


Figure 9:

Here the picture becomes much clearer again, at the same time that the memory bandwidth takes a sharp drop, the compute throughput does to.

5.3 Conclusion

6 Part 3: Roofline Model

6.1 Setup

The array size is fixed for all benchmark variations performed in this part to 2^{22} , based on the data gathered in part 1 and the workgroup size is set to 64.

6.2 Model

7 Part 4: Local vs Global

8 Extra: Desktop vs Macbook

9 Appendix

9.1 Part 1

GPU Performance Variability Analysis
(Coefficient of Variation across 15 Runs)

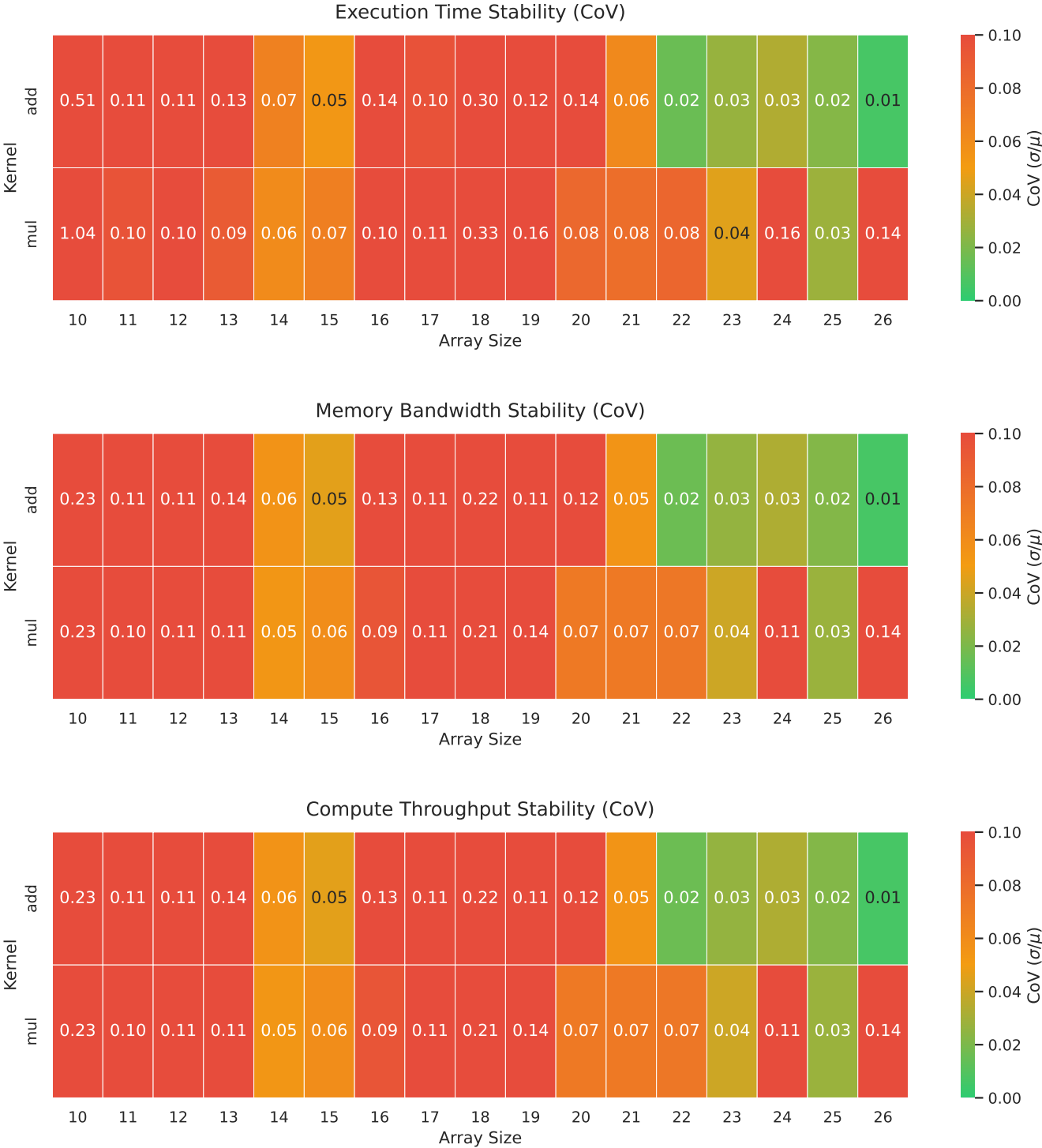


Figure 10:

9.2 Part 2

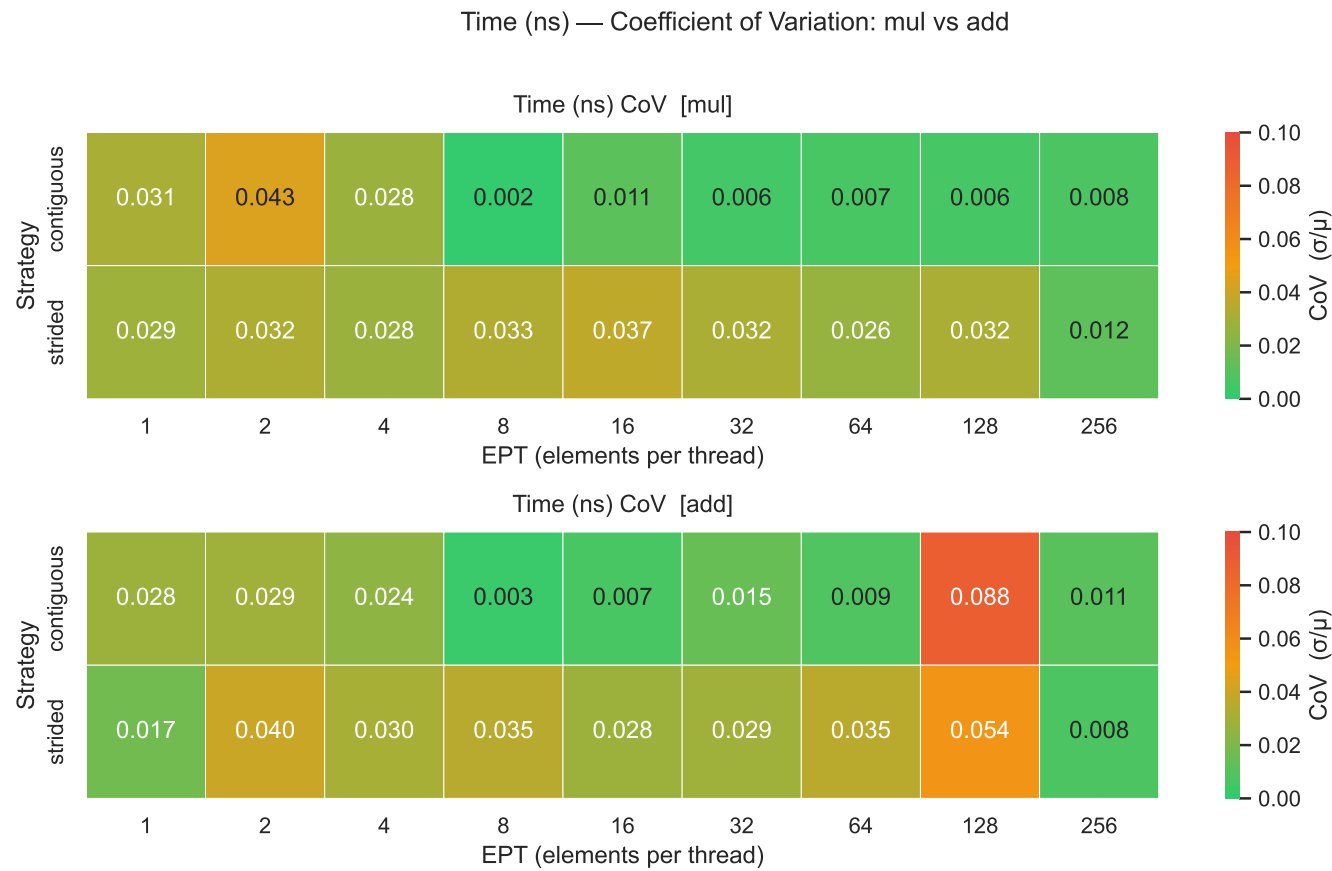


Figure 11:

9.3 Part 3

9.4 Specifications

9.4.1 Macbook

Part	Value
CPU	M2 Pro (6 performance and 4 efficiency)
RAM	16GB
OS	TODO

Table 1: Macbook Specifications

9.4.2 Desktop

Part	Value
CPU	Ryzen 9 5950X
GPU	RTX 3070
OpenCL	300 (Set in code)
Driver Version	591.59
RAM	64GB (3200 Mhz)
OS	Windows 11 - Version 10.0.22631 Build 22631

Table 2: Desktop Specifications

Bibliography

- [1] P. A. Paolillo, “Lecture 2 - Metrics, Distributions & Experimental Rigor - How Many runs do I need?.” p. 38, 2025.
- [2] P. A. Paolillo, “Lecture 2 - Metrics, Distributions & Experimental Rigor - Coefficient of Variation (CoV).” p. 25, 2025.